

EMBEDDED APPLICATIONS

Real Time Monitoring System

Malik Bogonluri

Inhoud

1. BESCHRIJVING.....	3
2. HARDWARE (DE SIMULATIE).....	3
2.1 De Sensorknooppunten (ESP32 Nodes).....	3
2.2 Het Controlecentrum (Raspberry Pi 4).....	3
2.3 De Actoren & Statuslogica.....	4
3. SCHEMA & CIRCUIT ONTWERP.....	5
3.1 Architectuur van het Schema.....	5
3.2 Componentenlijst (BOM).....	6
4. Het Bashboard.....	6
5. Programmacode.....	8
6. Datasheet.....	16

1. BESCHRIJVING

In een fabriek is het super duur als een machine zomaar stopt. Als één belangrijk onderdeel kapotgaat, ligt meteen de hele productie stil. Met mijn project heb ik een slimme bewaker gemaakt die dit voorkomt. Het systeem gebruikt sensoren om constant de hitte en vibraties van de machine te meten. Het doel is om te voorspellen wanneer er iets kapotgaat, zodat je het al kunt maken vóóordat het echt misgaat. Dit noemen we Preventief Onderhoud.

2. HARDWARE (DE SIMULATIE)

We gebruiken ESP32's als slimme knooppunten voor de draadloze dataverzameling. Door specifieke sensoren te gebruiken, simuleren we real-life industriële scenario's op een kleine, hanteerbare schaal:

2.1 De Sensorknooppunten (ESP32 Nodes)

We gebruiken ESP32's als slimme knooppunten voor de draadloze dataverzameling. Door specifieke sensoren te gebruiken, simuleren we real-life industriële scenario's op een kleine, hanteerbare schaal:

- **BME280 (Vibratie Simulatie):** Deze sensor meet temperatuur en luchtdruk. In onze logica worden stijgingen in deze waarden gebruikt om mechanische **vibraties** en wrijving te simuleren.
- **DHT11 (Omgevingsbewaking):** Deze sensor bewaakt de algemene temperatuur en luchtvochtigheid in de machinekamer om oververhitting en droogte (brandgevaar) te detecteren.
- **Communicatie:** De ESP32's sturen hun metingen via het MQTT-protocol over Wi-Fi naar de centrale server.

2.2 Het Controlecentrum (Raspberry Pi 4)

De Raspberry Pi 4 fungeert als de centrale "brain" van het project. Hierop draait de volledige backend-stack:

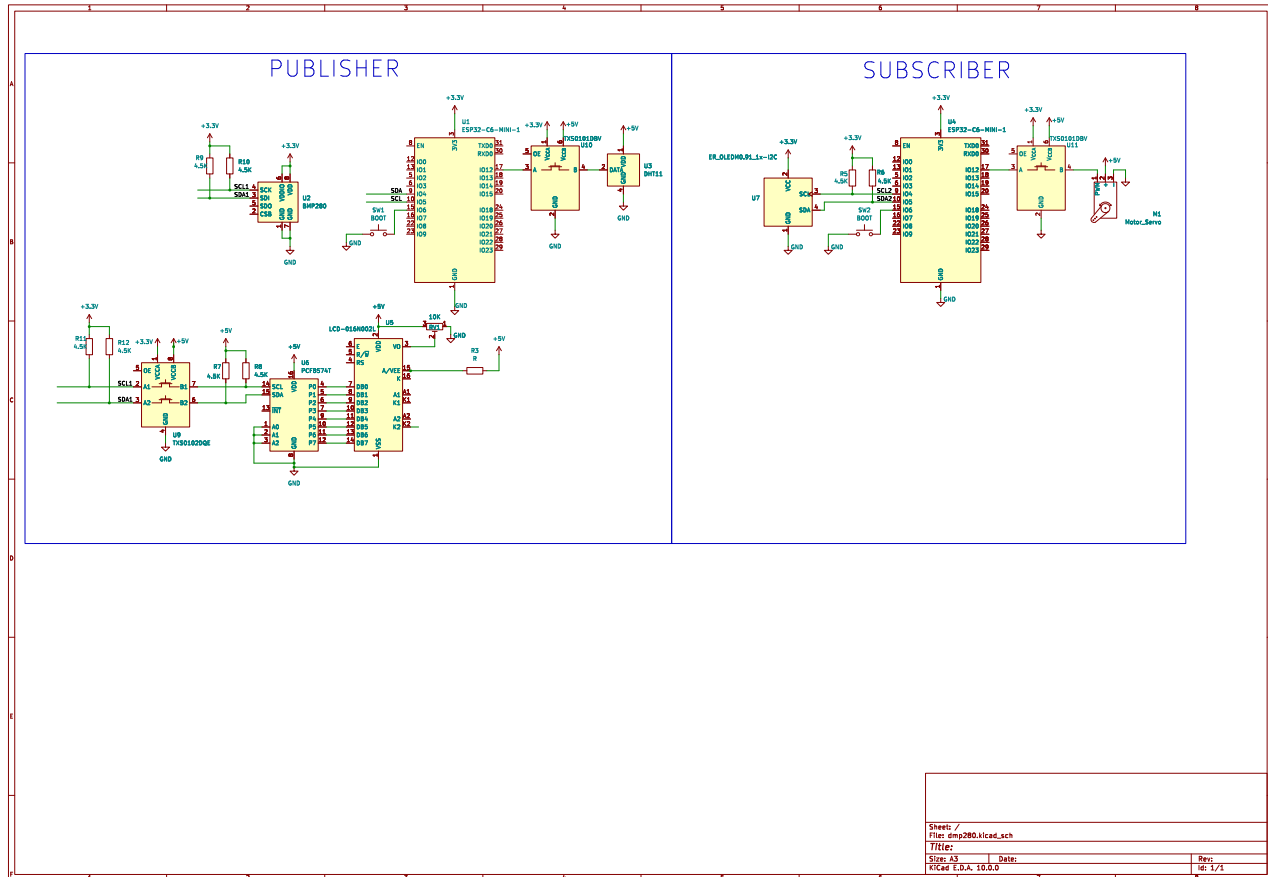
- **MQTT Broker (Mosquitto):** Vangt alle inkomende berichten van de sensoren op.
- **Database (InfluxDB):** Een tijdserie-database die meetwaarden razendsnel opslaat voor trendanalyse.
- **Visualisatie (Grafana):** Zet ruwe cijfers om in dashboards, waardoor patronen en afwijkingen direct zichtbaar worden.

2.3 De Actoren & Statuslogica

Het systeem is geprogrammeerd om zelfstandig beslissingen te nemen op basis van de binnengekomen data:

- **Systeem OK:** BME280 (vibratie) < 38.0°C en DHT11 (omgeving) < 35.0°C. De machine draait soepel.
- **Waarschuwing (Warning):** BME280 > 38.0°C OF DHT11 > 35.0°C. De servo draait naar 90° en het OLED-scherm toont **WARNING**. Dit duidt op beginnende slijtage.
- **Kritiek (Critical):** BME280 > 50.0°C EN luchtvochtigheid < 15%. De **Servo-motor** draait naar 180° (simulatie van een **fysieke noodstop**) en het scherm toont **CRITICAL**.
- **Fail-safe:** Indien er langer dan **7 seconden** geen data wordt ontvangen, schakelt het systeem naar **NO DATA** (servo op 90°) om blind draaien te voorkomen.

3. SCHEMA & CIRCUIT ONTWERP



3.1 Architectuur van het Schema

Het ontwerp maakt gebruik van een modulaire opzet waarbij de ESP32-C6-Mini centraal staat:

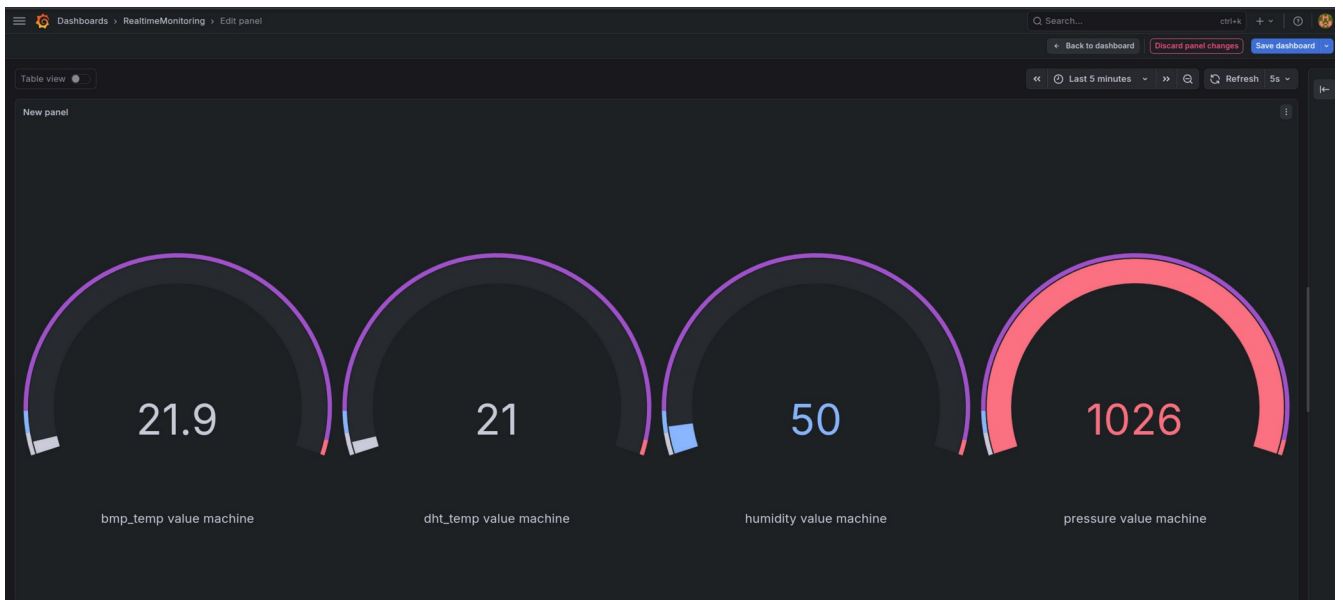
- **Logic Level Shifting (TXS0101/TXS0102):** Omdat de ESP32 op 3.3V werkt en de DHT11/Servo op 5V, garanderen deze shifters een veilige en betrouwbare signaaloverdracht.
- **I2C Bus Management:** De BME280 en het OLED-display delen de I2C-bus. Externe pull-up weerstanden van 4.7k Ω houden de communicatie stabiel.
- **Power Rails:** Gescheiden 5V (actuatoren) en 3.3V (sensoren) lijnen met een gemeenschappelijke massa (GND) voorkomen zwevende signalen en ruis.

3.2 Componentenlijst (BOM)

Component	Functie	Interface
ESP32-C6-Mini	Hoofdprocessor & WiFi/MQTT stack	Microcontroller
BME280	Vibratie simulatie (Temp/Druk)	I2C (0x76)
DHT11	Omgevingstemp & Luchtvochtigheid	Digital (via Level Shifter)
OLED SSD1306	Real-time status display (64x48)	I2C (0x3C)
TXS0101/02	Voltage vertaling 3.3V <-> 5V	Level Shifter
Servo Motor	Fysieke noodstop / Actuator	PWM

4. Het Dashboard

Om de machine op afstand in de gaten te kunnen houden, heb ik een dashboard gemaakt dat in een controlekamer gebruikt kan worden. In één oogopslag zie je hier de belangrijkste gegevens: de vibratie-simulatie (via de BME-temperatuur), de temperatuur van de machinekamer, de luchtvochtigheid en de luchtdruk. De gekleurde meters maken het heel simpel voor een operator; zolang de wijzers in het paars staan is alles veilig, maar zodra een meter rood kleurt, is er direct actie nodig. Zo kan de technische dienst vanuit hun kantoor precies zien of een machine nog soepel draait of dat er preventief onderhoud nodig is.



5. Programmacode

```
1  #include <WiFi.h>
2  #include <PubSubClient.h>
3  #include <Wire.h>
4  #include <Adafruit_GFX.h>
5  #include <Adafruit_SSD1306.h>
6  #include <ESP32Servo.h>
7
8  // --- OLED CONFIG ---
9  #define SCREEN_WIDTH 64
10 #define SCREEN_HEIGHT 48
11 #define OLED_RESET -1
12 Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
13
14 // --- SERVO CONFIG ---
15 Servo statusServo;
16 const int servoPin = 12;
17
18 // --- STATE TRACKING ---
19 enum SystemStatus { STATUS_OK, STATUS_WARNING, STATUS_CRITICAL, STATUS_NO_DATA, STATUS_INIT };
20 SystemStatus lastStatus = STATUS_INIT;
21
22 // --- THRESHOLDS ---
23 float vibCritical = 50.0;
24 float vibWarning = 38.0;
25 float dryLimit = 15.0;
26 unsigned long dataTimeout = 7000;
27
28 // --- NETWORK CONFIG ---
29 const char* ssid = "Orange-c45e0";
30 const char* password = "48LW8yBb";
31 const char* mqttServer = "192.168.0.9";
32
33 float curDHT=0, curBMP=0, curHum=0;
34 unsigned long lastMsgTime = 0;
35
36 WiFiClient espClient;
37 PubSubClient client(espClient);
38
39 // --- SEPARATE SERVO FUNCTION ----+
40 void triggerServo(int angle, String label) {
41   Serial.print(">>> STATUS CHANGE: ");
42   Serial.print(label);
43   Serial.print(" | Moving to: ");
44   Serial.println(angle);
45
46   statusServo.attach(servoPin, 544, 2400);
47
48   statusServo.write(angle);
49   delay(600); // Slightly longer delay to ensure full travel
50   statusServo.detach();
51 }
52
53
```



```

54 void updateDisplay(SystemStatus s) {
55     display.clearDisplay();
56     display.setTextSize(1);
57     display.setTextColor(WHITE);
58     display.setCursor(0, 4);
59
60     switch(s) {
61         case STATUS_NO_DATA: display.print(">> NO DATA"); break;
62         case STATUS_CRITICAL: display.print(">> CRITICAL"); break;
63         case STATUS_WARNING: display.print(">> WARNING"); break;
64         case STATUS_OK: display.print(">> SYS OK"); break;
65         default: display.print(">> BOOTING"); break;
66     }
67
68     display.drawFastHLine(0, 20, 64, WHITE);
69     display.display();
70 }
71
72 void checkStatusChange() {
73     SystemStatus currentStatus;
74
75     if (millis() - lastMsgTime > dataTimeout) {
76         currentStatus = STATUS_NO_DATA;
77     } else {
78         bool isCritical = (curBMP > vibCritical && curHum < dryLimit);
79         bool isWarning = (curBMP > vibWarning || curDHT > 35.0);
80
81         if (isCritical) currentStatus = STATUS_CRITICAL;
82         else if (isWarning) currentStatus = STATUS_WARNING;
83         else currentStatus = STATUS_OK;
84     }
85
86     if (currentStatus != lastStatus) {
87         lastStatus = currentStatus;
88         updateDisplay(currentStatus);
89
90         if (currentStatus == STATUS_CRITICAL) triggerServo(180, "CRITICAL");
91         else if (currentStatus == STATUS_WARNING) triggerServo(90, "WARNING");
92         else if (currentStatus == STATUS_OK) triggerServo(0, "SYS OK");
93         else if (currentStatus == STATUS_NO_DATA) triggerServo(90, "NO DATA");
94     }
95 }
96
97 void callback(char* topic, byte* payload, unsigned int length) {
98     lastMsgTime = millis();
99     String msg = "";
100     for (int i = 0; i < length; i++) msg += (char)payload[i];
101     float val = msg.toFloat();
102
103     if (strstr(topic, "dht_temp")) curDHT = val;
104     else if (strstr(topic, "bmp_temp")) curBMP = val;
105     else if (strstr(topic, "humidity")) curHum = val;
106
107     checkStatusChange();
108 }

```

```

109
110 void setup() {
111     Serial.begin(115200);
112     ESP32PWM::allocateTimer(0);
113     ESP32PWM::allocateTimer(1);
114
115     Wire.begin(21, 22);
116     if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { for(;;); }
117
118     updateDisplay(STATUS_INIT);
119
120     WiFi.begin(ssid, password);
121     while (WiFi.status() != WL_CONNECTED) {
122         delay(500);
123         Serial.print(".");
124     }
125
126     client.setServer(mqttServer, 1883);
127     client.setCallback(callback);
128     lastMsgTime = millis();
129 }
130
131 void reconnect() {
132     while (!client.connected()) {
133         String clientId = "ESP32_Vibe_Stat_" + String(random(0xffff), HEX);
134         if (client.connect(clientId.c_str())) {
135             client.subscribe("fabriek/machine/#");
136         } else {
137             delay(5000);
138         }
139     }
140 }
141
142 void loop() {
143     if (!client.connected()) reconnect();
144     client.loop();
145
146     static unsigned long lastCheck = 0;
147     if (millis() - lastCheck > 1000) {
148         lastCheck = millis();
149         checkStatusChange();
150     }
151 }

```

```

1  #include <WiFi.h>
2  #include <PubSubClient.h>
3  #include <Wire.h>
4  #include <Adafruit_BMP280.h>
5  #include <LiquidCrystal_I2C.h>
6  #include <Bonezegei_DHT11.h>
7
8  // WiFi & MQTT Config
9  const char* ssid = "Orange-c45e0";
10 const char* password = "48LW8yBb";
11 const char* mqttServer = "192.168.0.9";
12 const int mqttPort = 1883;
13 const char* mqttUser = "cherry";
14 const char* mqttPassword = "mqtt-test";
15 const char* clientID = "esp32-c6";
16
17 // Pin Config
18 #define SDA_PIN 4
19 #define SCL_PIN 5
20 #define DHT_PIN 9
21
22 WiFiClient espClient;
23 PubSubClient client(espClient);
24 Adafruit_BMP280 bmp;
25 LiquidCrystal_I2C lcd(0x27, 16, 2);
26 Bonezegei_DHT11 dht(DHT_PIN);
27
28 unsigned long lastMsg = 0;
29 bool showDHT = false; // Toggle for LCD screens
30
31 void setup() {
32     Serial.begin(115200);
33
34     // 1. Start I2C & DHT
35     Wire.begin(SDA_PIN, SCL_PIN);
36     dht.begin();
37     delay(2000);
38
39     // 2. Start LCD
40     lcd.init();
41     lcd.backlight();
42     lcd.clear();
43     lcd.print("System Booting...");
44
45     // 3. Start BMP280
46     if (!bmp.begin(0x76)) {
47         Serial.println("BMP280 Error!");
48         lcd.setCursor(0,1);
49         lcd.print("Sensor Error");
50         while (1);
51     }

```

```
53 // 4. Start WiFi
54 WiFi.begin(ssid, password);
55 while (WiFi.status() != WL_CONNECTED) {
56     delay(500);
57     Serial.print(".");
58 }
59
60 lcd.clear();
61 lcd.print("WiFi Connected");
62 client.setServer(mqttServer, mqttPort);
63 delay(1000);
64 }
65
66 void loop() {
67     if (!client.connected()) {
68         if (client.connect(clientID, mqttUser, mqttPassword)) {
69             Serial.println("MQTT Connected");
70         } else {
71             delay(5000);
72             return;
73         }
74     }
75     client.loop();
76
77     unsigned long now = millis();
78     if (now - lastMsg > 2000) {
79         lastMsg = now;
80
81         // Read BMP280
82         float bmpTemp = bmp.readTemperature();
83         float pressure = bmp.readPressure() / 100.0F;
84
85         // Read DHT11
86         float dhtTemp = 0;
87         int humidity = 0;
88         if (dht.getData()) {
89             dhtTemp = dht.getTemperature();
90             humidity = dht.getHumidity();
91         }
92     }
93 }
```

```

92
93 // --- LCD DISPLAY (Alternates every 2 seconds) ---
94 lcd.clear();
95 if (!showDHT) {
96     // Screen 1: BMP280 Data
97     lcd.setCursor(0, 0);
98     lcd.print("BMP Temp: "); lcd.print(bmpTemp, 1); lcd.print("C");
99     lcd.setCursor(0, 1);
100     lcd.print("Pres: "); lcd.print(pressure, 0); lcd.print(" hPa");
101     showDHT = true;
102 } else {
103     // Screen 2: DHT11 Data
104     lcd.setCursor(0, 0);
105     lcd.print("DHT Temp: "); lcd.print(dhtTemp, 1); lcd.print("C");
106     lcd.setCursor(0, 1);
107     lcd.print("Humidity: "); lcd.print(humidity); lcd.print("%");
108     showDHT = false;
109 }
110
111 // --- SEND MQTT ---
112 client.publish("fabriek/machine/bmp_temp", String(bmpTemp).c_str());
113 client.publish("fabriek/machine/pressure", String(pressure).c_str());
114 client.publish("fabriek/machine/dht_temp", String(dhtTemp).c_str());
115 client.publish("fabriek/machine/humidity", String(humidity).c_str());
116
117 // Serial Monitor Backup
118 Serial.print("BMP:"); Serial.print(bmpTemp);
119 Serial.print(" | DHT:"); Serial.print(dhtTemp);
120 Serial.print(" | Hum:"); Serial.println(humidity);
121 }
122 }
123

```

```

Read from remote host 192.168.69.12: Connection reset by peer
Connection to 192.168.69.12 closed.
client_loop: send disconnect: Broken pipe
cobalt@quart:~$ ssh cherry@192.168.69.12
from influxdb_client import InfluxDBClient, Point
from influxdb_client.client.write_api import SYNCHRONOUS

# --- InfluxDB 2.x Configuration ---
INFLUXDB_URL = "http://192.168.0.9:8086"
INFLUXDB_TOKEN = "tRkLfjW2m0SqZMgJitIEE24N_VtDCyF9Tq2Emg7JKjvhms0Es9EK2x0GAPZoQsklhF2X6qX4k-UuvDDFo1gIxg=="
INFLUXDB_ORG = "Thomas More"
INFLUXDB_BUCKET = "HomeAutomation"

# --- MQTT Configuration ---
MQTT_ADDRESS = '192.168.0.9'
MQTT_USER = 'cherry'
MQTT_PASSWORD = 'mqtt-test'
MQTT_TOPIC = 'fabriek/+/+' # Changed to match your Regex pattern
MQTT_REGEX = r'fabriek/([^\s]+)/([^\s/\s]+)'
#MQTT_CLIENT_ID = 'clientId-QQw10x8zHk'
MQTT_CLIENT_ID = 'MQTTInfluxDBBridge'

# Create InfluxDB 2.x client
influx_client = InfluxDBClient(url=INFLUXDB_URL, token=INFLUXDB_TOKEN, org=INFLUXDB_ORG)
write_api = influx_client.write_api(write_options=SYNCHRONOUS)

```

```

class SensorData(NamedTuple):
    location: str
    measurement: str
    value: float

def on_connect(client, userdata, flags, rc, properties=None):
    print(f"Connected to MQTT with result code: {rc}")
    client.subscribe(MQTT_TOPIC)

def on_message(client, userdata, msg):
    payload = msg.payload.decode('utf-8')
    print(f"MQTT Received: {msg.topic} -> {payload}")

    match = re.match(MQTT_REGEX, msg.topic)
    if match:
        location = match.group(1)
        measurement = match.group(2)
        try:
            val = float(payload)
            # Create InfluxDB Point
            point = Point(measurement) \
                .tag("location", location) \
                .field("value", val)

            write_api.write(bucket=INFLUXDB_BUCKET, org=INFLUXDB_ORG, record=point)
            print(f"✅ Data written to InfluxDB: {measurement} from {location}")
        except ValueError:
            print("Payload is not a number")
    else:
        print("Topic did not match regex")

```



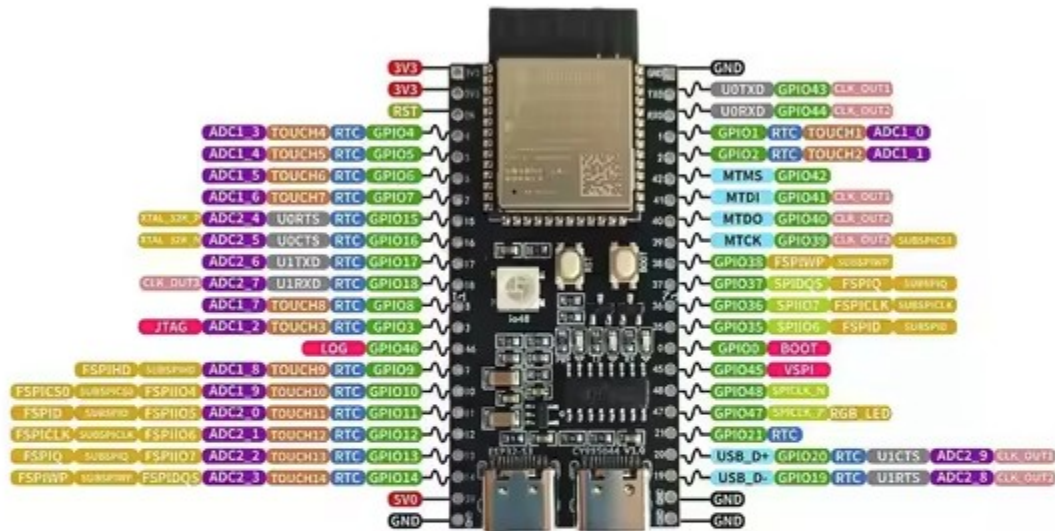
```
def main():
    print("Starting MQTT to InfluxDB bridge...")

    mqtt_client = mqtt.Client(client_id=MQTT_CLIENT_ID)
    mqtt_client.username_pw_set(MQTT_USER, MQTT_PASSWORD)
    mqtt_client.on_connect = on_connect
    mqtt_client.on_message = on_message

    try:
        mqtt_client.connect(MQTT_ADDRESS, 1883, 60)
        mqtt_client.loop_forever()
    except KeyboardInterrupt:
        print("Stopping...")
    finally:
        influx_client.close()

if __name__ == "__main__":
    main()
```

6. Datasheet



ESP32-S3 Specs

32-bit Xtensa® dual-core @240MHz
 Wi-Fi IEEE 802.11 b/g/n 2.4GHz + BLE 5 Mesh
 512 KB SRAM (16 KB SRAM in RTC)
 384 KB ROM
 45 GPIOs, 4x SPI, 3x UART, 2x I2C,
 14x Touch, 2x I2S, RMT, LED PWM, USB-OTG,
 TWAI®, 2x 12-bit ADC, 1x LCD interface, DVP

MISC Miscellaneous/SPI functions
 CLK_OUT2 Clock Output

PWM Capable Pin
 GPIOX GPIO Input and Output
 JTAG/USB JTAG for Debugging and USB
 ADCX_CH Analog-to-Digital Converter
 TOUCHX Touch Sensor Input Channel
 OTHER Other Related Functions
 SERIAL Serial for Debug/Programmin
 STRAP Strapping Pin Functions
 RTC RTC Power Domain (VDDIP3_RTC)
 GND Ground
 PWD Power Rails (3V3 and 5V)

ESP32-C6-MINI-1

ESP32-C6-MINI-1U

Datasheet Version 1.4

Module that supports 2.4 GHz Wi-Fi 6 (802.11ax), Bluetooth® 5 (LE), Zigbee and Thread (802.15.4)

Built around ESP32-C6 series of SoCs, 32-bit RISC-V single-core microprocessor

Flash up to 8 MB in chip package

22 GPIOs, rich set of peripherals

On-board PCB antenna or external antenna connector



ESP32-C6-MINI-1



ESP32-C6-MINI-1U

3.1 Pin Layout

The pin diagram below shows the approximate location of pins on the module. For the actual diagram drawn to scale, please refer to Figure 10.1 *Module Dimensions*.

The pin diagram is applicable for ESP32-C6-MINI-1 and ESP32-C6-MINI-1U, but the latter has no keepout zone.

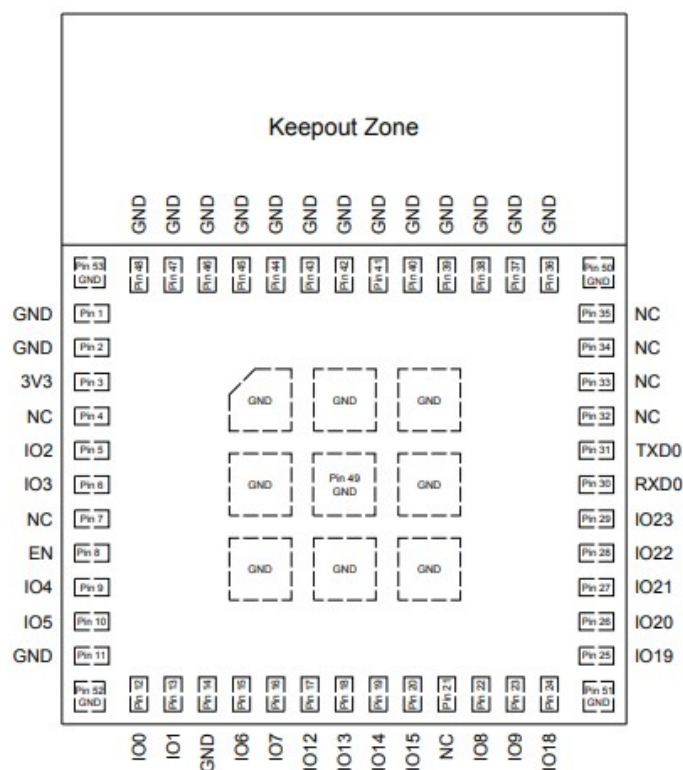


Figure 3-1. Pin Layout (Top View)

3.2 Pin Description

The module has 53 pins. See pin definitions in Table 3-1 *Pin Definitions*.

For peripheral pin configurations, please refer to [ESP32-C6 Series Datasheet](#).

Table 3-1. Pin Definitions

Table 3-1 – cont'd from previous page

Name	No.	Type ¹	Function
NC	4	—	NC
IO2	5	I/O/T	GPIO2, LP_GPIO2, LP_UART_RTSN, ADC1_CH2, FSPIQ
IO3	6	I/O/T	GPIO3, LP_GPIO3, LP_UART_CTSN, ADC1_CH3
NC	7	—	NC
EN	8	I	High: on, enables the chip. Low: off, the chip powers off. Note: Do not leave the EN pin floating.
IO4	9	I/O/T	MTMS, GPIO4, LP_GPIO4, LP_UART_RXD, ADC1_CH4, FSPIHD
IO5	10	I/O/T	MTDI, GPIO5, LP_GPIO5, LP_UART_TXD, ADC1_CH5, FSPIWP
IO0	12	I/O/T	GPIO0, XTAL_32K_P, LP_GPIO0, LP_UART_DTRN, ADC1_CHO
IO1	13	I/O/T	GPIO1, XTAL_32K_N, LP_GPIO1, LP_UART_DSRN, ADC1_CH1
IO6	15	I/O/T	MTCK, GPIO6, LP_GPIO6, LP_I2C_SDA, ADC1_CH6, FSPICLK
IO7	16	I/O/T	MTDO, GPIO7, LP_GPIO7, LP_I2C_SCL, FSPID
IO12	17	I/O/T	GPIO12, USB_D-
IO13	18	I/O/T	GPIO13, USB_D+
IO14	19	I/O/T	GPIO14
IO15	20	I/O/T	GPIO15
NC	21	—	NC
IO8	22	I/O/T	GPIO8
IO9	23	I/O/T	GPIO9
IO18	24	I/O/T	GPIO18, SDIO_CMD, FSPICS2
IO19	25	I/O/T	GPIO19, SDIO_CLK, FSPICS3
IO20	26	I/O/T	GPIO20, SDIO_DATA0, FSPICS4
IO21	27	I/O/T	GPIO21, SDIO_DATA1, FSPICS5
IO22	28	I/O/T	GPIO22, SDIO_DATA2
IO23	29	I/O/T	GPIO23, SDIO_DATA3
RXD0	30	I/O/T	U0RXD, GPIO17, FSPICS1
TXD0	31	I/O/T	U0TXD, GPIO16, FSPICS0
NC	32	—	NC
NC	33	—	NC
NC	34	—	NC
NC	35	—	NC

¹ P: power supply; I: input; O: output; T: high impedance.

9 Peripheral Schematics

This is the typical application circuit of the module connected with peripheral components (for example, power supply, antenna, reset button, JTAG interface, and UART interface).

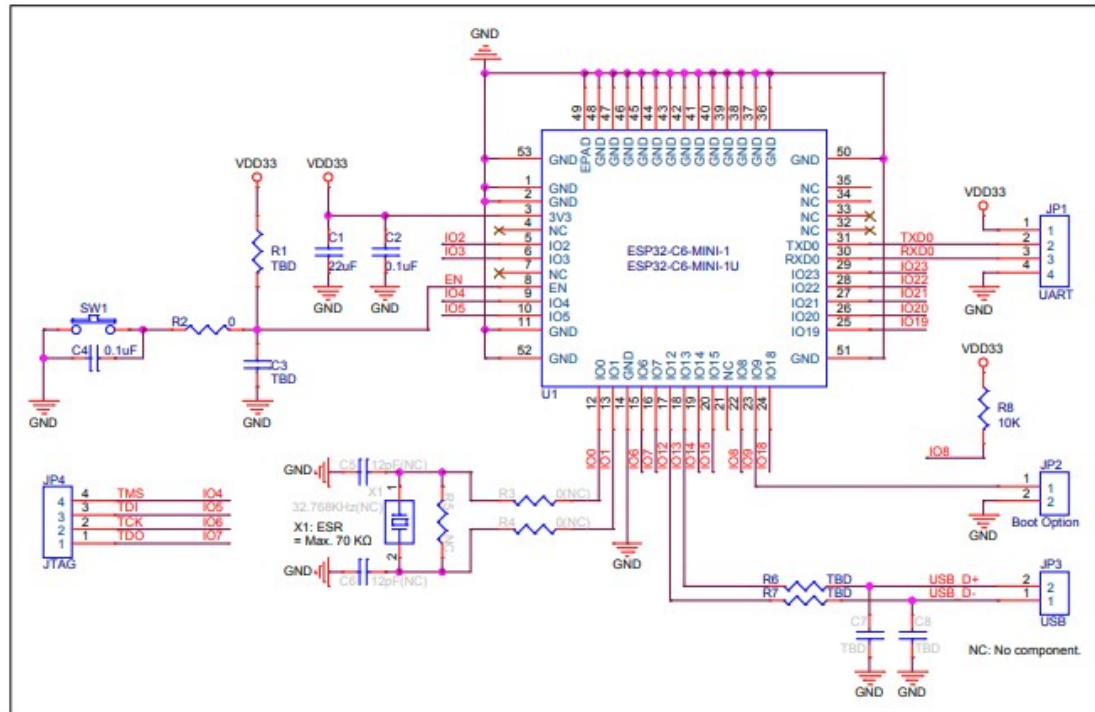
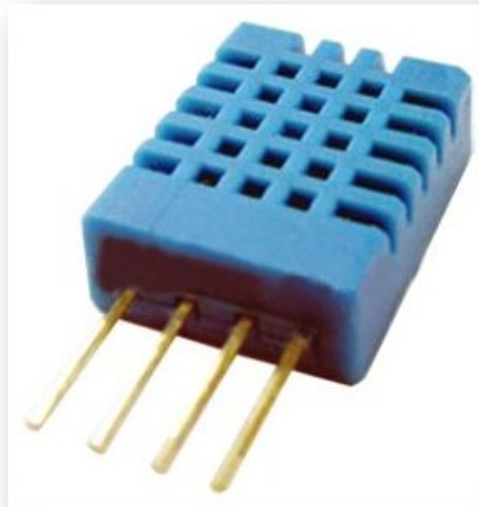


Figure 9-1. Peripheral Schematics

- Soldering the EPAD to the ground of the base board is not a must, however, it can optimize thermal performance. If you choose to solder it, please apply the correct amount of soldering paste. Too much soldering paste may increase the gap between the module and the baseboard. As a result, the adhesion between other pins and the baseboard may be poor.
- To ensure that the power supply to the ESP32-C6 chip is stable during power-up, it is advised to add an RC delay circuit at the EN pin. The recommended setting for the RC delay circuit is usually $R = 10\text{ k}\Omega$ and $C = 1\text{ }\mu\text{F}$. However, specific parameters should be adjusted based on the power-up timing of the module and the power-up and reset sequence timing of the chip. For ESP32-C6's power-up and reset sequence



Each DHT11 element is strictly calibrated in the laboratory that is extremely accurate on humidity calibration. The calibration coefficients are stored as programmes in the OTP memory, which are used by the sensor's internal signal detecting process. The single-wire serial interface makes system integration quick and easy. Its small size, low power consumption and up-to-20 meter signal transmission making it the best choice for various applications, including those most demanding ones. The component is 4-pin single row pin package. It is convenient to connect and special packages can be provided according to users' request.

2. Technical Specifications:

Overview:

Item	Measurement Range	Humidity Accuracy	Temperature Accuracy	Resolution	Package
DHT11	20-90%RH 0-50 °C	±5%RH	±2 °C	1	4 Pin Single Row

7.2 Connection diagram I²C

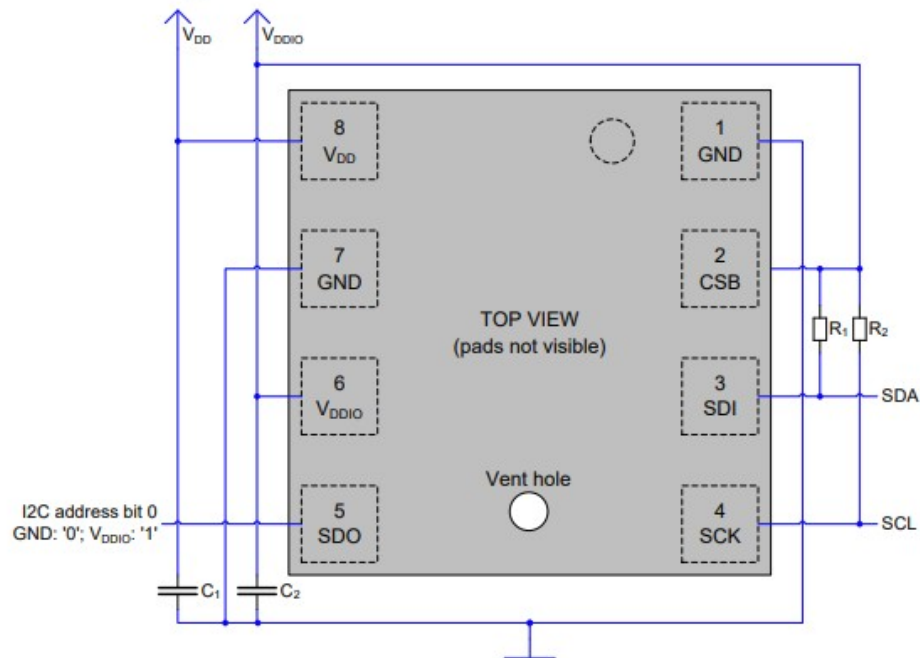


Figure 17: I²C connection diagram

Notes:

- The recommended value for C₁, C₂ is 100 nF
- The value for the pull-up resistors R₁, R₂ should be based on the interface timing and the bus load; a normal value is 4.7 kΩ
- A direct connection between CSB and V_{DDIO} is required



CONTACT

Naam | Functie
xxx.xxx@thomasmore.be
Tel. + 32 xx xx xx xx

VOLG ONS

www.thomasmore.be
fb.com/ThomasMoreBE
#WeAreMore

THOMAS
MORE